専門科目 :情報

Python によるデータ可視化

初めての人向け・90 分 ×8 コマ

長崎大学病院 医療情報部　薗田 光太郎

放送大学 長崎学習センター

2026 年 11 月 14 日(土)・15 日(日)

目次

はじめに 7
この講座で目指すこと 7
全 8 コマの構成 7
準備 8

第 I 部　1 日目 9

第 1 章　第 1 コマ　データから表へ、表からグラフへ（導入編） 11
1.1　この講座で目指すこと 11
1.2　次の 2 日間の流れ 11
1.3　例：平均が同じでも、姿はまったく違う 12
1.4　グラフが力を発揮する場面 12
1.5　逆に、グラフが向かない場面 12
1.6　演習：図を見て、姿を言葉にする（ペアで 40 分） 13
1.7　まとめ 13

第 2 章　第 2 コマ　Python 開発環境の構築 15
2.1　JupyterLab Desktop の準備 15
2.2　作業フォルダ（ワーキングディレクトリ）の設定 15
2.3　日本語化 16
2.4　JupyterLab の画面 17
2.5　基本の使い方：4 つのステップ 18
2.6　データのダウンロード（実習の前に） 18
2.7　最初の試し実行 19
2.8　まとめ 20

第 3 章　第 3 コマ　Python プログラミング、外部ライブラリの導入 21

本資料（画像を含む）の無断転載・無断使用は禁止します.

3.1 Python は「計算と指示」ができる電卓 . . . 21
3.2 変数：リテラルに付けた名札 . . . 21
3.3 リストと辞書 . . . 22
3.4 繰り返しと条件分岐 . . . 22
3.5 関数とライブラリ . . . 23
3.6 外部ライブラリの導入 . . . 23
3.7 演習 . . . 24
3.8 まとめ . . . 24

第 4 章 第 4 コマ　Pandas と Matplotlib を用いた Jupyter ノートブック 25
4.1 このコマの進め方：Jupyter ノートブックで表を作る . . . 25
4.2 表の 4 段階 . . . 25
4.3 段階 1：収集（一覧表） . . . 26
4.4 段階 2：要約（代表値・統計値の表） . . . 27
4.5 段階 3：関係（クロス集計表） . . . 27
4.6 段階 4：推測と混同行列 . . . 29
4.7 信号検出理論：刺激のあるなし × 反応のあるなし . . . 29
4.8 表を作るときの約束 . . . 30
4.9 演習 . . . 30
4.10 まとめ（1 日目の振り返り） . . . 30

第 II 部　2 日目 33

第 5 章 第 5 コマ　データ変数の尺度、Python 言語の変数型タイプ 35
5.1 データ集めの主な方法 . . . 35
5.2 「度量衡」という言葉について . . . 35
5.3 変数と尺度：データには「ものさし」がある . . . 35
5.4 尺度で「できること」が違う . . . 36
5.4.1 数字でも、名義尺度として扱うもの . . . 36
5.5 水準と選択肢 . . . 37
5.6 標本という考え方 . . . 37
5.7 尺度とグラフの相性（目安） . . . 37
5.8 実習：データを読み込んで、列の尺度を考える . . . 38
5.9 データの型：Python の変数型と pandas の dtype . . . 39
5.10 演習 . . . 40

5.11 まとめ 40

第 6 章 第 6 コマ　データから表へ、表からグラフへ（実践編） 41
6.1 グラフは「知りたいこと」で選ぶ 41
6.2 各グラフは「何の、どんな様子」を見せるか 41
6.3 pandas のグラフは「表から直接」 42
6.4 折れ線・棒・散布図 42
6.5 ヒストグラム：分布の形を見る 45
6.6 箱ひげ図：分布を要約して比べる 46
6.6.1 四分位数：データを小さい順に「4 つに分ける」 47
6.6.2 箱ひげ図の描き方 48
6.6.3 外れ値（ハズレ値）：飛び抜けた点の扱い 48
6.7 伝わるデザインの約束 49
6.8 並べて比べる 52
6.9 演習 53
6.10 前半のまとめ（グラフのいろいろ） 53
6.11 データ分析：気づきをグラフだけで納得させる 54
6.12 分析の進め方：眺める → 気づく → 確かめる →納得させる 54
6.13 比べるときは「分布の格差」で見る 54
6.14 実習データ：反応時間の実験（80 名） 54
6.15 「グラフだけで納得させる」ための工夫 56
6.16 演習 56
6.17 まとめ 56

第 7 章 第 7 コマ　データスクレイピングと図表作成 57
7.1 データスクレイピングとは 57
7.2 発展：年齢層の割合の推移 60
7.3 集めたデータの整え方と図表の作り方 63
7.4 実習 63
7.5 まとめ 64

第 8 章 第 8 コマ　まとめ 65
8.1 講座全体の振り返り 65
8.2 まとめの課題（テスト） 65
8.3 課題スターター 66
8.4 進め方の目安（第 6 コマの手順） 67

8.5 提出前チェックリスト 67
8.6 振り返り 67

付録 69

用語のミニ辞典 69

困ったときの対処表 71

はじめに

Python によるデータ可視化 ——初めての人向け・90 分 ×8 コマ

- 科目コード :専門科目 :情報
- 講師 :薗田 光太郎(長崎大学病院 医療情報部)
- 日程 : 2026 年 11 月 14 日(土)・15 日(日)

この講座で目指すこと

平均などの要約だけでは分からない「データの姿」を、グラフで納得できる形で知ることが、この講座の芯です。

1. 平均などの数字だけでは、データの姿は分からない
2. 表の読み方・作り方を知ると、グラフが読みやすくなる
3. グラフは「知りたいこと」に合わせて選ぶ
4. 最後は、自分の気づきをグラフだけで納得させられるようになる

全 8 コマの構成

コマ	テーマ
第 1 コマ	データから表へ、表からグラフへ(導入編)
第 2 コマ	Python 開発環境の構築
第 3 コマ	Python プログラミング、外部ライブラリの導入
第 4 コマ	Pandas と Matplotlib を用いた Jupyter ノートブック
第 5 コマ	データ変数の尺度、Python 言語の変数型タイプ
第 6 コマ	データから表へ、表からグラフへ(実践編)
第 7 コマ	データスクレイピングと図表作成

本資料(画像を含む)の無断転載・無断使用は禁止します.

コマ	テーマ
第 8 コマ	まとめ

準備

- 環境は **JupyterLab Desktop**（ノートブックを使うための無料ソフトで、Python 本体が一緒に入っています）です。インストール方法は第 2 コマで説明します。作業フォルダ mylab の設定と日本語化も第 2 コマで行います
- 実習データは data.zip をダウンロードし、展開して出てくる `data` フォルダを、作業フォルダ `mylab` の中に置いてください。ノートブックは `mylab` の直下に作ります（詳しくは第 2 コマ）
- 表とグラフには **pandas**、日本語表示には **matplotlib-fontja** を使います。入れ方は第 3 コマで案内します（ノートブックのセルで `%pip install` を実行します）

第Ⅰ部

1日目

第 1 章

第 1 コマ　データから表へ、表からグラフへ（導入編）

このコマの進め方 :講義（40 分） → 図を見て姿を言葉にする演習（40 分）→まとめ（10 分）。このコマではまだプログラムは書きません。第 2 コマから JupyterLab Desktop で手を動かします。

1.1　この講座で目指すこと

1. 平均などの数字だけでは、データの姿は分からない
2. 表の読み方・作り方を知ると、グラフが読みやすくなる
3. グラフは「知りたいこと」に合わせて選ぶ
4. 最後は、自分の気づきをグラフだけで納得させられるようになる

1.2　次の 2 日間の流れ

- 1 日目 :第 1 コマ 導入編 → 第 2 コマ Python 開発環境の構築 → 第 3 コマ Python プログラミング、外部ライブラリの導入 → 第 4 コマ Jupyter ノートブック（Pandas と Matplotlib）
- 2 日目 :第 5 コマ データ変数の尺度、Python 言語の変数型タイプ → 第 6 コマ実践編 → 第 7 コマ データスクレイピングと図表作成 → 第 8 コマ まとめ

本資料（画像を含む）の無断転載・無断使用は禁止します.

1.3　例：平均が同じでも、姿はまったく違う

3 つのクラスは、平均点がどれも約 65 点です。テストの点数をグラフ（ヒストグラム）にすると、姿はこんなに違います。

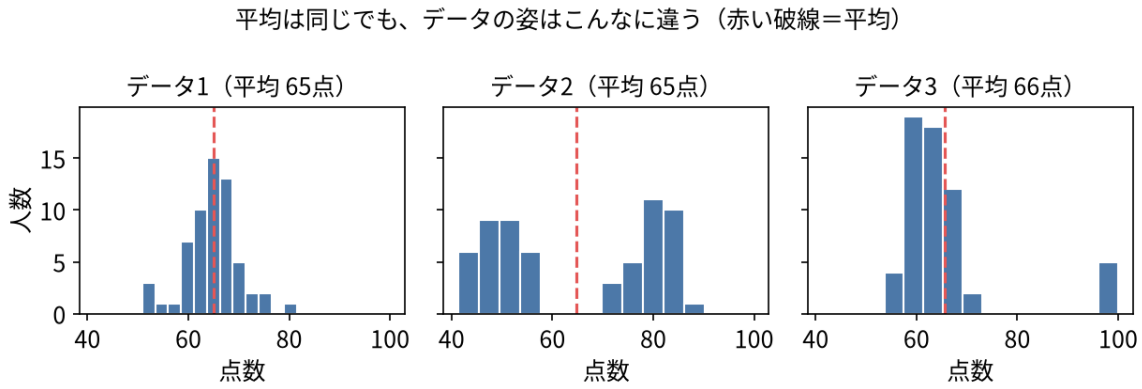


図 1.1: 平均が同じでも姿はまったく違う

平均（要約）だけ見ていては、山の高さ、広がり、偏りといった「姿」は分かりません。グラフで見るからこそ、姿が見えます。

> **ノート**
>
> この図は講師がプログラムで描いたものです。作り方は講座の中で少しずつ身につけます。

1.4　グラフが力を発揮する場面

- たくさんの数を一度に比べたいとき（表は縦に長いと読みにくい）
- 時間の流れや変化を見たいとき
- 「なんとなく多そう」という感覚を、人に納得させたいとき

1.5　逆に、グラフが向かない場面

- 正確な数値そのものを伝えたいとき（例：請求書の金額）は、表や数字のほうがよい
- データが数件しかないときは、数字をそのまま書くほうが伝わる

1.6 演習 :図を見て、姿を言葉にする（ペアで 40 分）

1. 配られた 3 種類の図（ヒストグラム／箱ひげ図／散布図）をペアで眺める
2. それぞれについて、「どんな姿か」を一文で書く。数字の暗記ではなく、言葉で表す練習
   - 例 :「真ん中に山が 1 つあって、右に長くすそを引いている」
   - 例 :「2 つのグループは重なっている部分が大きい」
3. ペアで読み合い、相手に伝わる表現だったかを確かめる

1.7 まとめ

- 平均などの要約だけでは、データの姿は分からない
- グラフは、姿を見せ、感覚を「納得」に変える道具
- ただし正確な数値には表の出番もある。使い分けが大事

次回は JupyterLab Desktop を準備して、最初のプログラムを実行します。

第 2 章

第 2 コマ　Python 開発環境の構築

このコマの進め方：JupyterLab Desktop の準備（作業フォルダと日本語化）と画面説明（25 分）→基本操作の実習とデータの配置（55 分）→ まとめ（10 分）。このコマでは、実習のための環境（JupyterLab Desktop とデータ）を整えます。

2.1　JupyterLab Desktop の準備

- JupyterLab Desktop は、ノートブック（コードと結果を同じ画面で見ながら進める形式）を使うための無料ソフトです。**Python 本体が一緒に入っている**ので、Python を別に入れる必要はありません
- Windows へのインストールは、ターミナル（コマンドプロンプト）で次を実行してください

```
winget install ProjectJupyter.JupyterLab
```

- うまくいかない場合は、公式の配布ページ（github.com/jupyterlab/jupyterlab-desktop の Releases）からインストーラーをダウンロードしてください

2.2　作業フォルダ（ワーキングディレクトリ）の設定

- まず、講義用の作業場所（ワーキングディレクトリ）となるフォルダを、ドキュメント（マイドキュメント）の中に作ります。名前は例として **mylab** とします
- JupyterLab Desktop を起動し、右上のハンバーガーメニュー（≡ ）から **Settings** を選び、**Default working directory** に、作った mylab のフォルダを指定します

本資料（画像を含む）の無断転載・無断使用は禁止します.

- これからの実習は、この mylab の中で進めます。ノートブックも実習データも、mylab の中に置きます

2.3　日本語化

- mylab の中に新しいノートブックを開き、セルで次を実行してください

```
%pip install jupyterlab-language-pack-ja-JP
```

- 実行が終わったら、JupyterLab を再起動します
- 再起動したら、メニューの **Settings → Language** を開き、「**Japanese (Japan) – 日本語(日本)**」を選びます
- 確認が出たら「**Change and reload**」を押します。画面が再読み込みされて、日本語表示になります
- 再起動だけでは日本語にはなりません。言語パックのインストールと、この言語の選択の両方が必要です
- なお、パッケージの導入(pandas・matplotlib-fontja)は第 3 コマで別に案内します。ここでは日本語化だけを行います

2.4 JupyterLab の画面

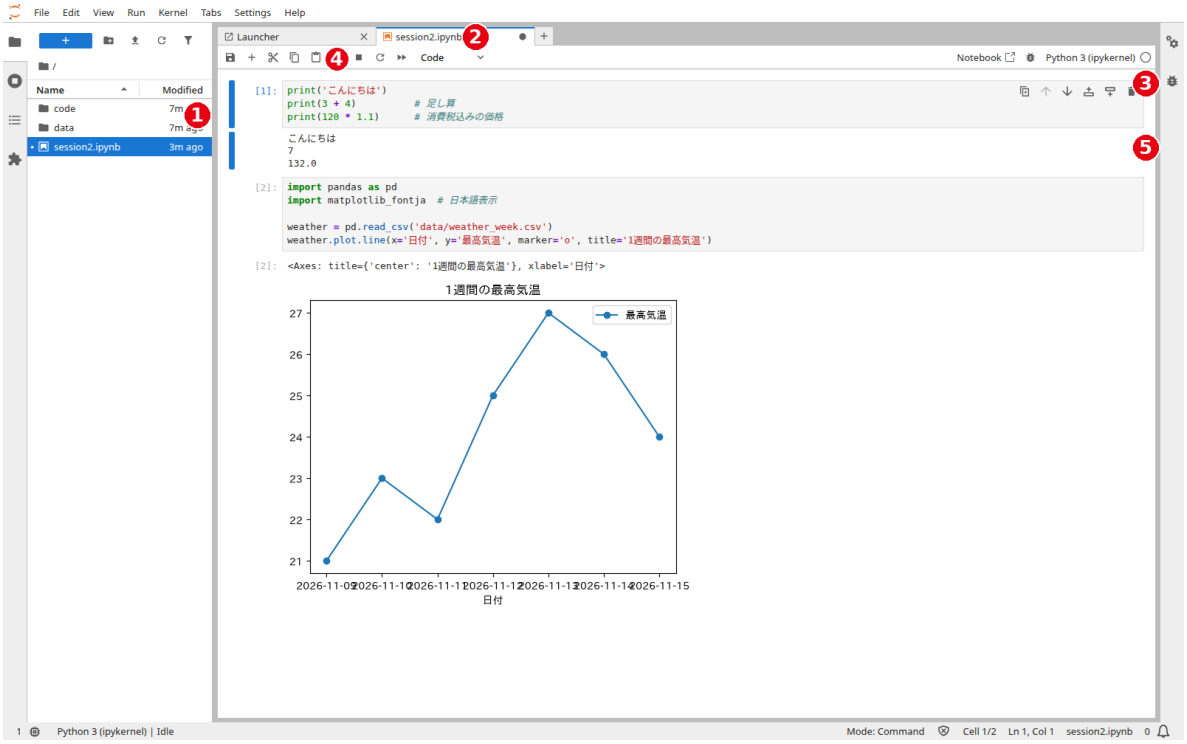


図 2.1: JupyterLab の画面（実物。番号は下の説明に対応）

1. **ファイルブラウザ**（左側）。開いたフォルダの中のフォルダとファイルの一覧です。ファイルを選んで開きます
2. **ランチャー／タブ**。ランチャーからは新しいノートブックを作れます。タブには、開いているノートブックの名前が出ます
3. **ノートブックのセル**。コードを書く欄です。セル（枠）ごとに区切って書きます
4. **実行ボタン**（▶）。選んでいるセルを実行します（キーボードの Shift+Enter でも同じです）
5. **出力**。実行結果やグラフは、セルのすぐ下に表示されます

（上の図は実際の画面です。JupyterLab Desktop でも同じ画面構成です。なお、この図は日本語化する前の画面のため、メニューが英語になっています。日本語化しても画面の構成は同じです）

2.5　基本の使い方：4 つのステップ

1. **セルに書く**：セル(コードを書く欄)にプログラムを書く
2. **実行する**：Shift+Enter（または実行ボタン▶)。結果がセルの下に出て、次のセルに進みます
3. **セルを足す**：続きは新しいセルに書く(ツールバーの「＋」ボタン)。セルごとに実行できるので、少しずつ試せます
4. **保存する**：Ctrl+S。ノートブックのファイルの末尾は`.ipynb` になります(例：`session2.ipynb`)

エラー(赤い文字)が出たら、セルの下に表示されるメッセージの**一番下の行**がヒントです。グラフも、セルの下にそのまま表示されます。

2.6　データのダウンロード(実習の前に)

- 実習で使うデータは、あらかじめ固めてある **data.zip** をダウンロードしてください
- 配布ページ(data.zip)：https://github.com/helmenov/lecture-dataviz/blob/main/data.zip
- ページ右側のダウンロードボタンで落とし、**展開(解凍)すると data フォルダが出てきます**
- **置き場所**：作業フォルダ **mylab** の中に置きます

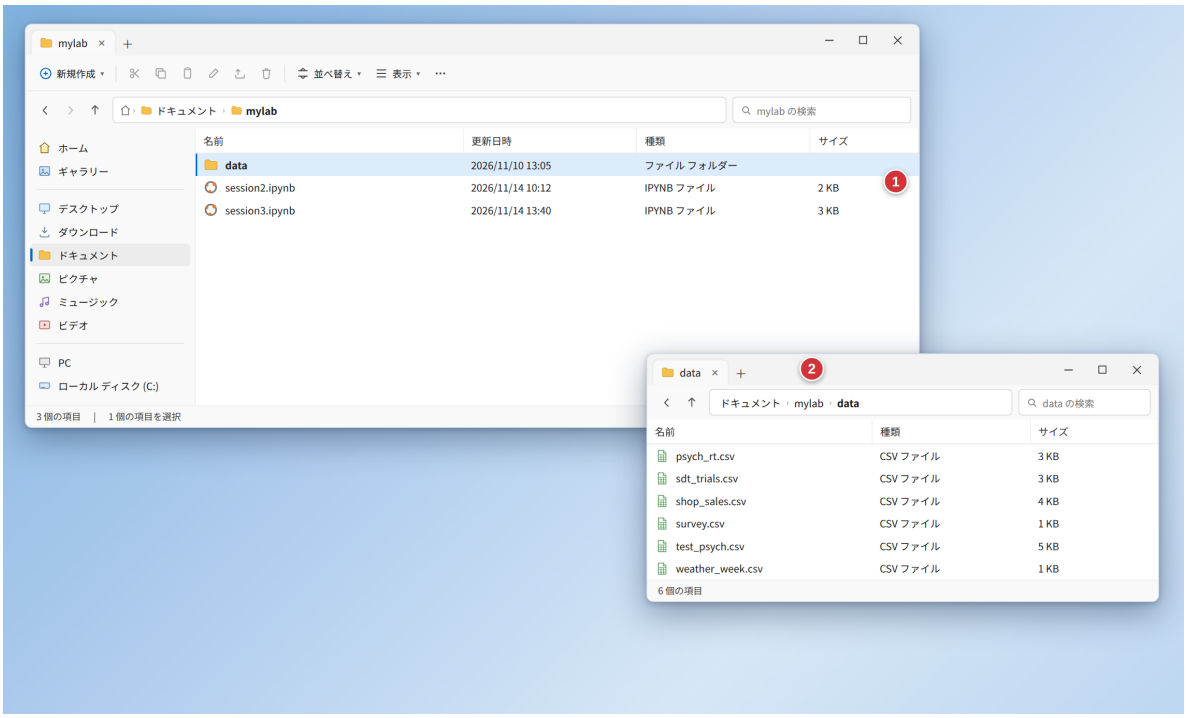


図 2.2: 置き場所のイメージ(エクスプローラーで mylab を開いたところ)

1. **data フォルダ**(赤い丸 1)。ダウンロードした data.zip を展開して出てくるフォルダです。mylab の中に置きます
2. **data フォルダの中身**(赤い丸 2)。開くと、実習で使う csv ファイルが入っています

(この図は、エクスプローラーの画面を再現したイメージ図です。実際の画面と細部が少し違うことがあります。ノートブック(session2.ipynb など)は、自分で mylab の直下に作ります)

- ノートブックは mylab の**直下**に作ってください。プログラムは、同じフォルダの中の data フォルダからデータを読みます。**置く場所を変えると、データが見つからずエラーになります**
- zip で配られている場合は、展開(解凍)してから置いてください。ダウンロードフォルダに置いたままにしないこと
- テスト用のデータ(test_psych.csv)も data フォルダに入っています。第 8 コマで使います

2.7 最初の試し実行

環境が整ったか、4 つのステップで短いプログラムをセルで実行して確かめましょう。

```
print('こんにちは')
print(3 + 4)          # 足し算
print(120 * 1.1)      # 消費税込みの価格
```

```
こんにちは
7
132.0
```

（本の中では、このようなプログラムの実行結果もその場で表示されます）

2.8　まとめ

- JupyterLab Desktop の準備ができました。操作は「セルに書く → Shift+Enter で実行する → セルを足して続きを書く →保存する」の 4 ステップです
- 作業フォルダは mylab です。日本語化は、言語パックのインストールと、Settings → Language での「Japanese (Japan) – 日本語（日本）」の選択の両方が必要です
- 実習データ（data フォルダ）は mylab の中に置き、ノートブックは mylab の直下に作ります。置く場所を変えるとエラーになります
- 次のコマでは、プログラムの基本（変数・リスト・繰り返しなど）を学びます

第 3 章

第 3 コマ　Python プログラミング、外部ライブラリの導入

このコマの進め方 :講義と一緒に実習(70 分) → 演習(10 分) →まとめ(10 分)。第 2 コマで整えた JupyterLab Desktop のノートブックで、プログラムの基本を学びます。

3.1　Python は「計算と指示」ができる電卓

- プログラムは、上から順に実行されます
- # の後ろはコメント(説明書き)で、実行されません
- 第 2 コマで試した `print()` で、計算の答えを表示できます
  - 引数 (丸括弧の中) には リテラルや変数を入力します.

リテラル: カッコ付き文字列, カッコ無しの数値 ・数式

3.2　変数 : リテラルに付けた名札

カッコ無しの文字列で表現します.(カッコありの文字列は「リテラル」)まず定義があってから使用できるようになります.名札が数値の別名となります.

本資料(画像を含む)の無断転載 ・無断使用は禁止します.

```
price = 120           # リテラル数値の 120 で変数(名札) price を定義
tax = 1.1
print(price * tax)    # 箱の中身を使って計算する
price = 150           # 定義し直すこともできる
print(price * tax)
```

```
132.0
165.0
```

3.3　リストと辞書

```
scores = [72, 65, 80, 91]   # リスト: 並んだデータ
print(scores)                 # 表示
print(scores[2])              # 2番目のデータ
print(len(scores))          # 個数
print(sum(scores) / len(scores))  # 平均

person = {"名前": "田中", "点数": 72}  # 辞書: 名前付きのデータ
print(person["名前"], person["点数"])
```

```
[72, 65, 80, 91]
80
4
77.0
田中 72
```

3.4　繰り返しと条件分岐

```
for s in [72, 65, 80, 91]:   # リストの中身を1つずつ取り出す
    if s >= 80:
        print(s, "よくできました")
    else:
        print(s, "もう少し")
```

```
72 もう少し
65 もう少し
80 よくできました
91 よくできました
```

3.5 関数とライブラリ

- よく使う処理には名前が付いていて、呼び出すだけです（例：`print`、`len`、`sum`）
- 追加の道具箱が「ライブラリ」。`import` で読み込んで使います
- この講座では、表とグラフの道具箱 **pandas** を中心に使います

3.6 外部ライブラリの導入

- 追加の部品（pandas や matplotlib-fontja）は、ノートブックのセルで `%pip install` を実行して入れます
- この講座で使う **pandas**（表とグラフ）と **matplotlib-fontja**（日本語表示）、それに第 7 コマのスクレイピングで表を読み込むための **lxml** を、ここで入れておきましょう。第 4 コマ以降で使います

```
%pip install pandas matplotlib-fontja lxml
```

- 入れた後は、カーネルを再起動しないと使えないことがあります（メニューの「Kernel → Restart Kernel」）。再起動すると、それまでに作った変数は全部消えるので、上のセルから実行し直してください
- 第 7 コマで使う `requests`（ウェブページを取りに行く道具）は、通常は最初から入っています。もし「無い」というエラーが出たら、`%pip install requests` も実行してください

3.7　演習

1. 自分の名前と、今日の日付を `print` で表示する
2. リストに家族の年齢を入れて、平均を出す
3. 80 点以上なら「合格」、それ以外は「再挑戦」と表示するプログラムに書き換える

3.8　まとめ

- プログラムは上から順に実行される。結果はセルの下に出る
- 変数・リスト・繰り返し・条件分岐が分かれば、実習のコードは読める
- 道具箱（ライブラリ）を読み込むと、表とグラフが扱える

第 4 章

第 4 コマ　Pandas と Matplotlib を用いた Jupyter ノートブック

このコマの進め方 :講義(40 分) → 一緒に実習(35 分) →演習と振り返り(15 分)。表は「段階」ごとに姿が違います。収集 → 要約 →関係 → 推測、の 4 段階で整理します。

4.1　このコマの進め方：Jupyter ノートブックで表を作る

- このコマは、**Jupyter** ノートブックで **pandas** を使って表を作ります
- ノートブックでは、プログラムとその結果(表)を同じ画面で見ながら進められます。表の表示には `display()` を使います
- グラフ描画の裏側では **Matplotlib** が動きます。この後のコマでグラフを描くときに文字が欠けないよう、日本語表示(matplotlib-fontja)の準備も、上のプログラムで済ませてあります

4.2　表の 4 段階

段階	表の姿	すること
1 :収集	一覧表	集めたデータを、そのまま並べる
2 :要約	変数ごとに代表値・統計値に置き換えた表	平均・中央値などの代表値や統計値で置き換える
3 :関係	クロス集計表(分割表)	2 つの変数の組み合わせで数える

本資料(画像を含む)の無断転載・無断使用は禁止します.

段階	表の姿	すること
4 :推測	混同行列	推測の正誤の数を数える（行と列が同じカテゴリ）

4.3　段階 1 :収集(一覧表)

```
weather = pd.read_csv("data/weather_week.csv")
display(weather.head(10))
```

	日付	曜日	最高気温	最低気温	降水量
0	2026-11-09	月	21	13	0.0
1	2026-11-10	火	23	14	0.0
2	2026-11-11	水	22	15	3.5
3	2026-11-12	木	25	16	0.0
4	2026-11-13	金	27	17	0.0
5	2026-11-14	土	26	18	12.0
6	2026-11-15	日	24	15	0.5

- pandas では、表全体を **DataFrame** と呼びます
- まず全体を表示し、行数(データの件数)と列(変数)を確認します

```
display(weather.shape)      # (行数, 列数)
display(weather.dtypes)     # 各列の型
```

```
(7, 5)
```

```
日付          str
曜日          str
最高気温      int64
最低気温      int64
降水量      float64
dtype: object
```

4.4 段階 2：要約（代表値・統計値の表）

一覧表のままでは姿が見えないので、変数ごとに代表値や統計値に置き換えます。

```
rt = pd.read_csv("data/psych_rt.csv")
display(rt[["反応時間", "正答数"]].describe())
```

	反応時間	正答数
count	80.000000	80.000000
mean	378.625000	16.150000
std	98.511115	2.038242
min	243.000000	11.000000
25%	327.750000	15.000000
50%	370.000000	16.000000
75%	400.250000	18.000000
max	837.000000	20.000000

- `describe()` は、個数・平均・標準偏差・最小・四分位数・最大を一度に並べてくれます
- ここの四分位数（25%、50%、75%）が、第 6 コマの箱ひげ図につながります

4.5 段階 3：関係（クロス集計表）

```
survey = pd.read_csv("data/survey.csv")
display(pd.crosstab(survey["年代"], survey["満足度"]))
```

満足度 年代	1	2	3	4	5
10 代	3	3	1	0	8
20 代	2	2	3	3	1
30 代	2	0	1	4	0

40 代	2	2	3	2	2
50 代	1	3	1	4	2
60 代以上	2	0	1	1	1

- 割合で見たいときは `normalize="index"` を付けます(行ごとに 100%)

```
ct = pd.crosstab(survey["年代"], survey["満足度"],
 ↪  normalize="index")
display((ct * 100).round(1))
```

満足度 年代	1	2	3	4	5
10 代	20.0	20.0	6.7	0.0	53.3
20 代	18.2	18.2	27.3	27.3	9.1
30 代	28.6	0.0	14.3	57.1	0.0
40 代	18.2	18.2	27.3	18.2	18.2
50 代	9.1	27.3	9.1	36.4	18.2
60 代以上	40.0	0.0	20.0	20.0	20.0

- グループごとの合計や平均は `groupby` で作ります(この例は「一元表」にもなります)

```
sales = pd.read_csv("data/shop_sales.csv")
sales["金額"] = sales["単価"] * sales["個数"]
by_item = (sales.groupby("商品")[["金額", "個数"]].sum()
           .rename(columns={"金額": "金額合計", "個数": "個数合計"})
           .sort_values("金額合計", ascending=False))
display(by_item)
```

商品	金額合計	個数合計
コーヒー	541800	1204
紅茶	292400	731
ジュース	193200	552
ケーキ	189750	345

4.6 段階 4 :推測と混同行列

- 推測とは、あるデータの値から別のデータの値を当てることです。当てた結果の正しさを数えるのが、この段階の仕事です
- 推測の出来を評価する表が**混同行列**です。クロス集計表に似ていますが、**行と列が同じカテゴリ**（実際 × 判断）なのが特徴で、斜めのマスが正解です
- 心理学の**信号検出理論**が、その代表例です（次へ）

4.7 信号検出理論 :刺激のあるなし × 反応のあるなし

かすかな音が「聞こえたか」を答える実験のように、刺激があるかないかと、反応（「あった」と答えたか）の組み合わせで、判断の様子を調べる理論です。

実際 ＼ 反応	「あった」	「なかった」
刺激あり（50 試行）	38 **ヒット**	12 **見落とし**
刺激なし（50 試行）	9 **誤警報**	41 **正棄却**

- この人の的中の様子 :刺激ありでは 76% 当て、刺激なしでは 18%「あった」と言ってしまう
- 判断の基準がゆるい人は「ヒットも誤警報も多い」、厳しい人は「見落としが多い」といった具合に、4 つのマスのバランスに、その人の判断のクセが表れます
- この表はまさに混同行列です。pandas では `pd.crosstab(刺激の列, 反応の列)` で作れます

```
sdt = pd.read_csv("data/sdt_trials.csv")
cm = pd.crosstab(sdt["刺激"], sdt["反応"])
display(cm)
hit_rate = cm.loc["あり", "あった"] / cm.loc["あり"].sum()
fa_rate = cm.loc["なし", "あった"] / cm.loc["なし"].sum()
display(f"ヒット率: {hit_rate:.0%} / 誤警報率: {fa_rate:.0%}")
```

反応 刺激	あった	なかった
あり	38	12
なし	9	41

```
'ヒット率: 76% / 誤警報率: 18%'
```

4.8　表を作るときの約束

- タイトルと単位を付ける。「何の表か」が一目で分かること
- 数値の桁をそろえる。小数点以下の桁数は列ごとに統一する
- 出典と件数（n ＝何件か）を添える

4.9　演習

1. survey.csv で、年代ごとの満足度クロス集計を「割合」の表にし、一文で読み取る
2. shop_sales.csv で、商品ごとの金額合計の一元表を作る（単価 × 個数＝金額の列を足してから合計）
3. sdt_trials.csv の混同行列から、ヒット率と誤警報率を求める

4.10　まとめ（1 日目の振り返り）

- 表は 4 段階：収集（一覧表）／要約（代表値の表）／関係（クロス集計）／推測（混同行列）
- pandas では read_csv → head/shape → describe/crosstab/groupby が基本の流

れ

- 2 日目は、尺度の話のあと、この表をグラフに変えていきます

第 II 部

2 日目

第 5 章

第 5 コマ　データ変数の尺度、Python 言語の変数型タイプ

このコマの進め方 :講義(45 分) → 一緒に実習(30 分) →演習と振り返り(15 分)。

5.1　データ集めの主な方法

- **調査(アンケート)** :聞いて集める。質問の書き方で答えが変わる点に注意
- **観察・記録** :出来事をそのまま記録する。測り方をそろえることが大事
- **実験** :条件を変えて結果を比べる。比べる相手(対照)を用意する
- **既存データ** :公的な統計や、業務でたまったデータを使う。元の集め方を確認する

5.2　「度量衡」という言葉について

このコマで出てくる「度量衡(どりょうこう)」は、昔からの「測るための共通のものさし」のことです。

- 度(ど)＝長さ、量(りょう)＝かさ(体積)、衡(こう)＝重さ
- 今で言えば、メートル・リットル・グラムといった**単位の体系そのもの**
- この後の「比尺度」の正体は、この度量衡です。「誰がどこで測っても同じになる単位で測った数値」とつながります

5.3　変数と尺度 :データには「ものさし」がある

- 変数とは、測ったり記録したりする項目のこと(身長、満足度、血液型、気温…)

本資料(画像を含む)の無断転載・無断使用は禁止します.

- 尺度とは、その変数がどんな性質の「ものさし」で測られているか、という区別です
- 尺度が違うと、**やってよい計算や、合うグラフが変わります**

尺度	例	イメージ
名義尺度	血液型、商品名、天気	名札。区別だけ
順序尺度	順位、混雑具合、言葉の満足度（不満～満足）	順番はあるが間隔は不ぞろい
間隔尺度	気温（℃）、西暦、数値化した満足度	間隔は均等。0 は「無」ではない
比尺度	身長、金額、個数	度量衡（長さ・重さなどの共通単位）。0 ＝本当に無い

5.4　尺度で「できること」が違う

尺度	数える・割合	順位	差を言う（較べる）	倍率を言う（比べる）	平均
名義	できる	できない	できない	できない	できない
順序	できる	できる	できない	できない	注意
間隔	できる	できる	できる	できない	できる
比	できる	できる	できる	できる	できる

- 差を言う場合（気温）：較べる。5 ℃差と 5 ℃差は同じ
- 倍率を言う場合（重さ）：比べる。「2 倍重い」が言える
- 間隔尺度では、倍率は言えません（20 ℃は 10 ℃の「2 倍暑い」ではない）
- 度量衡でない数値データ（満足度など感覚を数値化したもの）も、**間隔尺度として扱います**。満足度は平均も使いますが、「4 点は 2 点の 2 倍満足」とは言えません

5.4.1　数字でも、名義尺度として扱うもの

- 背番号、郵便番号、商品コードのように、順序の意味（大小・強弱・優劣）がない数値は、**数字符号**として名義尺度扱いです

5.5 水準と選択肢

- **水準**：変数が取りうる値のこと。天気なら「晴れ・くもり・雨」が水準です
- **選択肢**：調査で答えてもらうために用意する水準の並び
- 選択肢の約束：もれがない（「その他」を用意）、重ならない、順序のあるものは順序を保つ
- 文字列の水準でも順序があれば順序尺度（例：順位、混雑具合、言葉の満足度）。数値に割り振り直せば、間隔尺度として扱ってよい

5.6 標本という考え方

- 本当に知りたい全体を**母集団**、実際に集めたデータを**標本（サンプル）**と言います
- 講座のデータも、たいていは標本です（例：来場者全員ではなく、協力してくれた 80 名）
- 結論はまず、「この標本では、こう見えた」と言える範囲に置くのが安全です
- 標本の偏り（若い人ばかり、協力的で元気な人ばかり等）があると、全体の姿とずれることがあります

5.7 尺度とグラフの相性（目安）

尺度	合うグラフ	例
名義	棒グラフ（人数・割合）	血液型ごとの人数
順序	棒グラフ（人数・割合）	満足度ごとの人数
間隔・比	ヒストグラム／箱ひげ図／折れ線／散布図	身長の分布、気温の変化

読み取りのポイント：**名義・順序は「人数を数えて棒グラフ」。平均の棒グラフにはしません。**間隔・比は分布を見るヒストグラムや箱ひげ図が使えます。

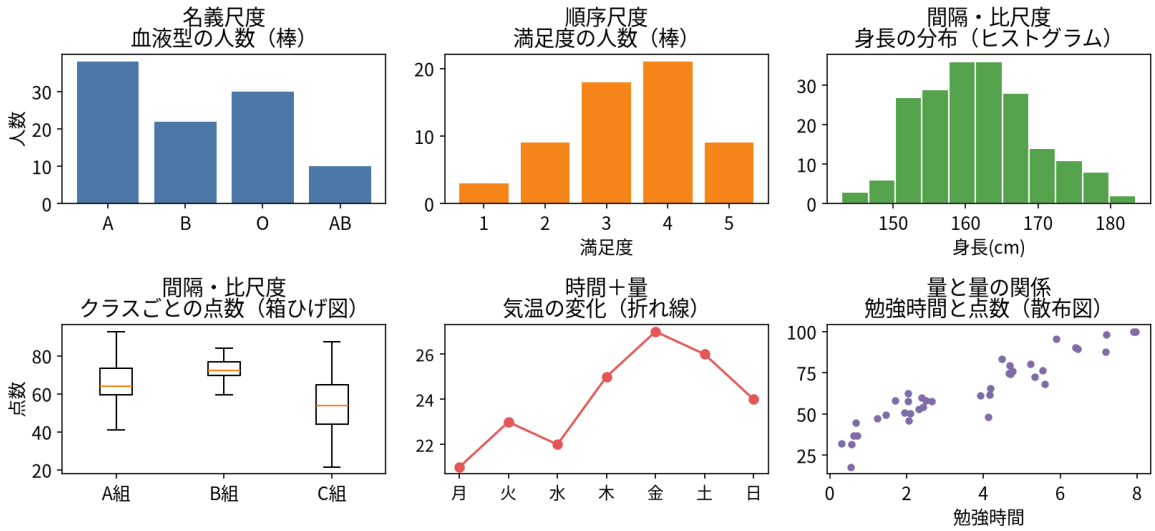


図 5.1: 尺度ごとの例となるグラフ

5.8　実習：データを読み込んで、列の尺度を考える

- pandas で読み込んだデータは，`print()` ではなく `display()` の方が読みやすい（整形されます）.

```
survey = pd.read_csv("data/survey.csv")
display(survey.head())
display(survey["年代"].unique())          # 水準を取り出す
display(survey["満足度"].value_counts())
```

	年代	満足度	来店回数
0	20 代	5	16
1	10 代	5	10
2	40 代	1	3
3	30 代	3	7
4	10 代	2	18

```
<StringArray>
['20 代', '10 代', '40 代', '30 代', '50 代', '60 代以上']
Length: 6, dtype: str

満足度
5    14
4    14
1    12
3    10
2    10
Name: count, dtype: int64
```

列ごとに「これは何尺度か」を言えるようにしておきましょう。

5.9 データの型：Python の変数型と pandas の dtype

- Python の変数には「型」があります。第 3 コマで見たように、数値のリテラルと「カッコ付き文字列」のリテラルは、別の型として扱われます
- pandas では列ごとに型が決まっていて、**dtype** と呼びます。第 4 コマで `display(weather.dtypes)` としたときに出てきたのが、各列の dtype です

```
display(survey.dtypes)    # 各列の型(dtype)
```

```
年代          str
満足度       int64
来店回数      int64
dtype: object
```

- 文字列の列は object、整数の列は int64、小数を含む列は float64 のように表示されます
- 型が違うと、できることが違います。文字列の列には平均は出せません。逆に、数値の列でも、数字符号（名義尺度）のような列は、平均しても意味がありません
- 尺度と dtype は、どちらも「その列で何をしてよいか」を教えてくれる手がかりです

5.10　演習

1. 身の回りのデータを 3 つ挙げ、それぞれの尺度を答える
2. 次のデータはどの尺度か :背番号、気温、血液型、満足度（1〜5 点）、身長、順位
   - 背番号 :名義（数字符号）。気温 :間隔。満足度（数値）:間隔として扱う。順位 :順序
3. アンケートの選択肢を 1 つ作り、「もれ・重なり・順序」の点検をする

5.11　まとめ

- 尺度は 4 つ（名義・順序・間隔・比）。比尺度の正体は度量衡
- 尺度によって、できる計算と合うグラフが決まる
- データは標本。結論は「この標本ではこう見えた」から始める

第 6 章

第 6 コマ　データから表へ、表からグラフへ（実践編）

このコマの進め方：前半はグラフのいろいろと伝わるデザイン（講義と実習）、後半は反応時間データを使ったデータ分析（講師デモと実習・演習）です。グラフは pandas の plot で描きます（裏側では matplotlib が動いています）。表示と仕上げだけ matplotlib を使います。日本語表示は matplotlib-fontja です。

6.1　グラフは「知りたいこと」で選ぶ

知りたいこと	合うグラフ
時間とともにどう変わったか	折れ線グラフ
量を比べたい（名義・順序）	棒グラフ
2 つの量に関係があるか	散布図
ばらつき・偏りを見たい	ヒストグラム
複数のグループの分布を比べたい	箱ひげ図
全体の中の割合	円グラフ／ 100% 積み上げ棒（注意あり）

第 5 コマの尺度とセットで考えます。「格好いいから」ではなく「質問に答える形」を選ぶのが原則です。

6.2　各グラフは「何の、どんな様子」を見せるか

本資料（画像を含む）の無断転載・無断使用は禁止します.

グラフ	何の、どんな様子を見せたいときに使うか
棒グラフ	カテゴリごとの**量の大きさ**。「どのカテゴリが、どれくらい大きいか」の様子
ヒストグラム	1 つの量の**分布の形**。「どこに集まり、どう偏り、山はいくつか、飛び抜けた値はないか」の様子
箱ひげ図	複数のグループの**分布の要約**。「中心はどこか、広がりはどうか、外れ値はあるか」をグループごとに比べる様子
折れ線グラフ	時間とともに量が**どう変化したか**。「増えた・減った・横ばいか、ピークはいつか」の様子
散布図	2 つの量の**関係**。「一方が大きいとき、もう一方も大きいか（右上がり／右下がり／無関係）、点はどう集まり、外れ値はないか」の様子

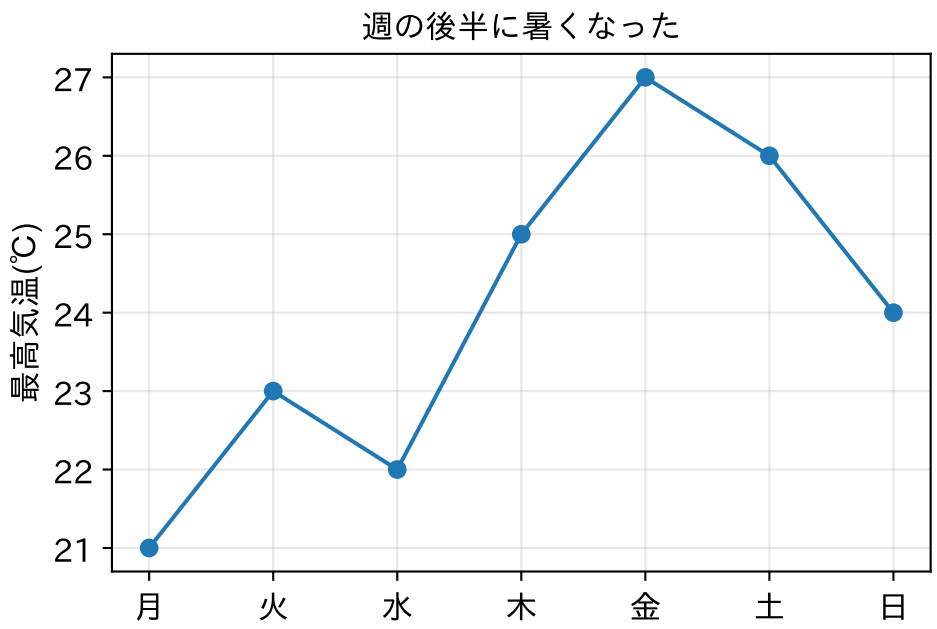
描く前に「何の、どんな様子を見せたいのか」を一言で言えることが、グラフ選びの出発点です。

6.3　pandas のグラフは「表から直接」

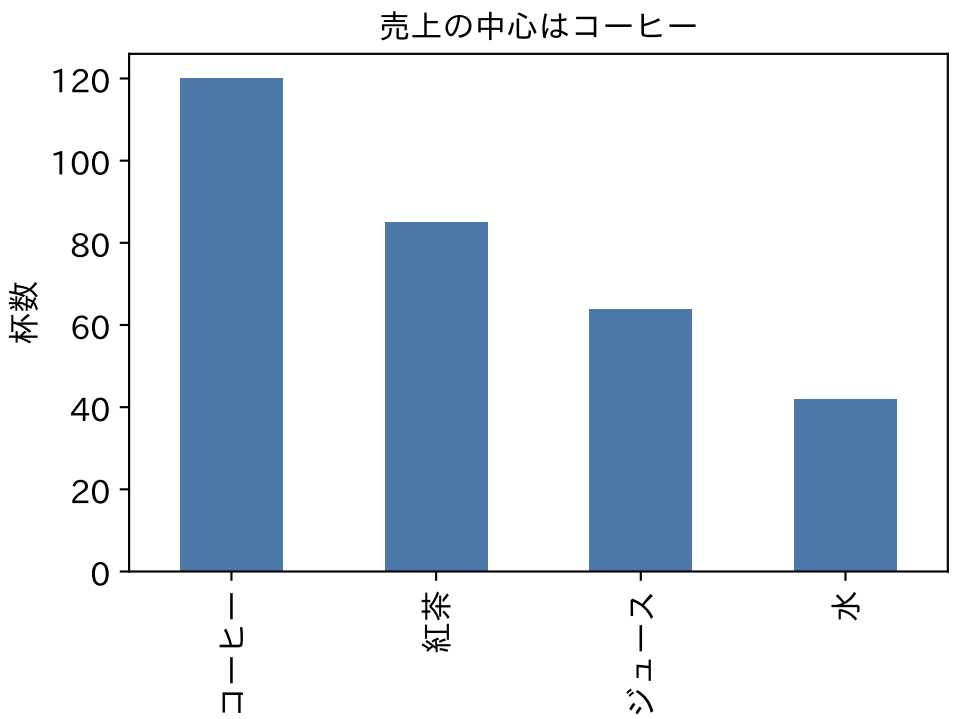
表（DataFrame）の `.plot` に種類（line、bar、scatter など）を指定するだけです。仕上げは matplotlib が担います。

6.4　折れ線・棒・散布図

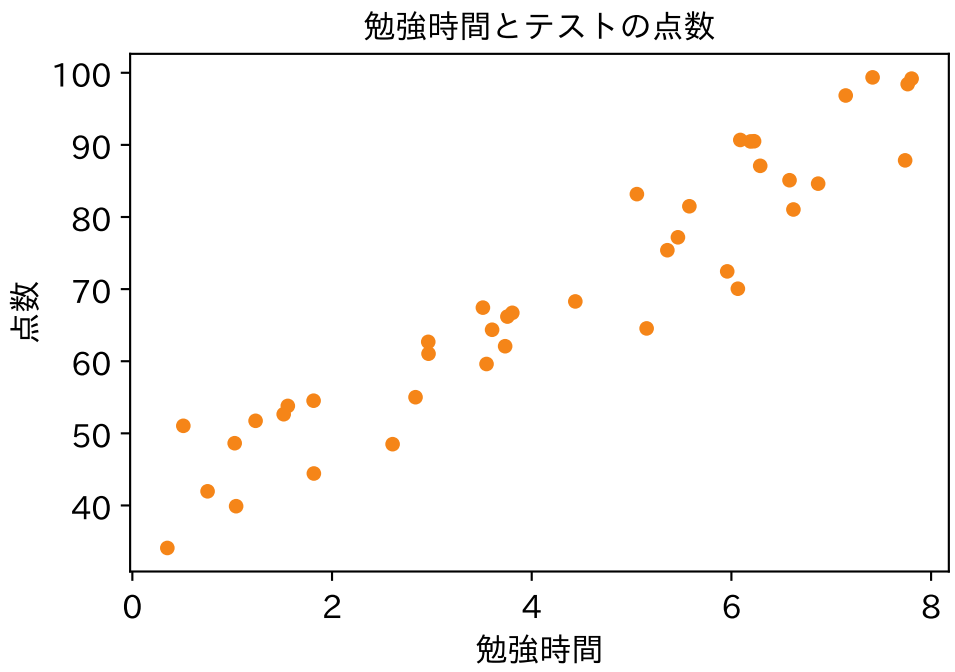
```
temp.plot.line(marker="o", title="週の後半に暑くなった", ylabel="最高
 ↪  気温（℃）")
plt.grid(alpha=0.3)
plt.show()
```



```
sales.plot.bar(color="#4C78A8", title="売上の中心はコーヒー",
 ↪  ylabel="杯数")
plt.show()
```




```
study.plot.scatter(x="勉強時間", y="点数", color="#F58518",
                   title="勉強時間とテストの点数")
plt.show()
```

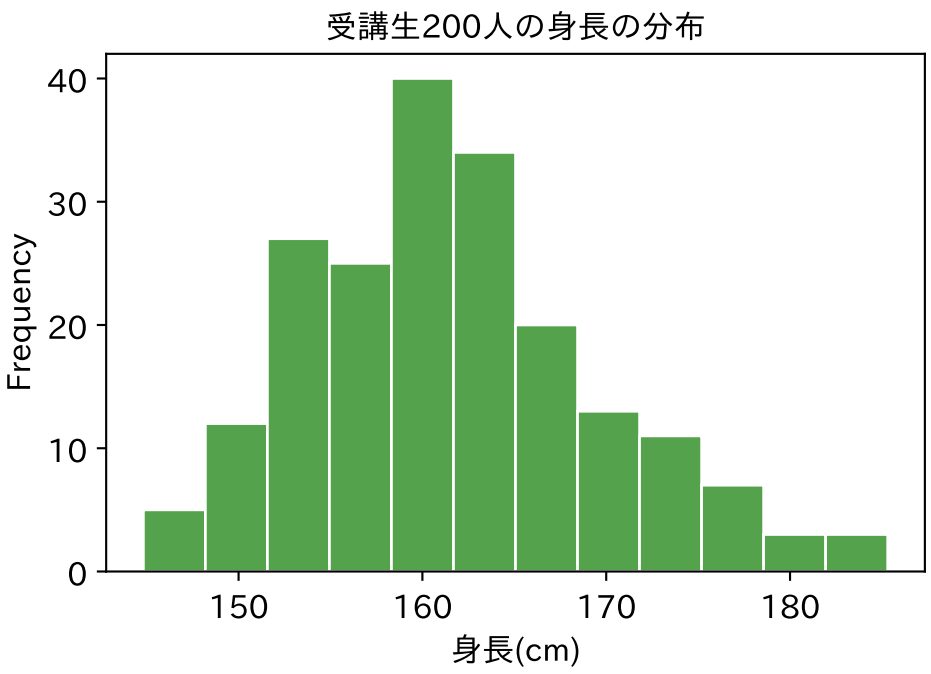


ノート

円グラフは角度が読みにくいので注意が必要です。3D グラフは、奥行きで値が読みにくいのでこの講座では扱いません。

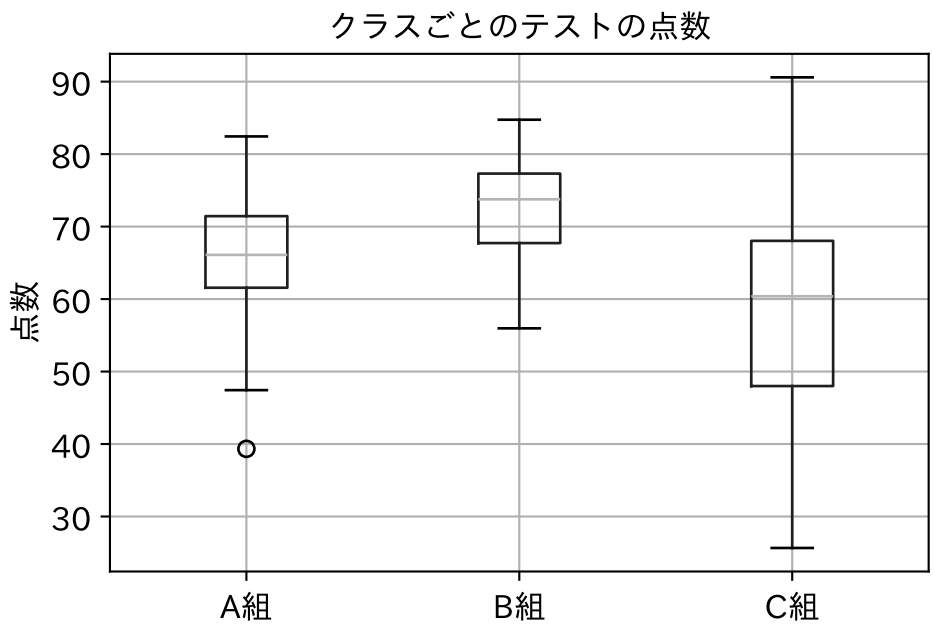
6.5 ヒストグラム：分布の形を見る

```
heights.plot.hist(bins=12, color="#54A24B", edgecolor="white",
                  title="受講生200人の身長の分布", xlabel="身長(cm)")
plt.show()
```



6.6　箱ひげ図：分布を要約して比べる

```
scores.boxplot()
plt.title("クラスごとのテストの点数")
plt.ylabel("点数")
plt.show()
```



6.6.1 四分位数：データを小さい順に「4 つに分ける」

箱ひげ図の中身は「四分位数」です。データ全体の真ん中 50% が箱に入ります。

- データを小さい順に並べたとき、下から 25% の位置が**第 1 四分位数（Q1）**、50% が**中央値**、75% が**第 3 四分位数（Q3）**
- 箱の幅 Q3 − Q1 を**四分位範囲（IQR）**と呼びます。真ん中 50% の広がりです
- `describe()` の 25%・50%・75%（第 4 コマ）に出ていた、あの行がそのまま箱の上下と中央の線になります
- なお、位置の決め方には流儀があり、手計算とソフトで少しずれることがあります

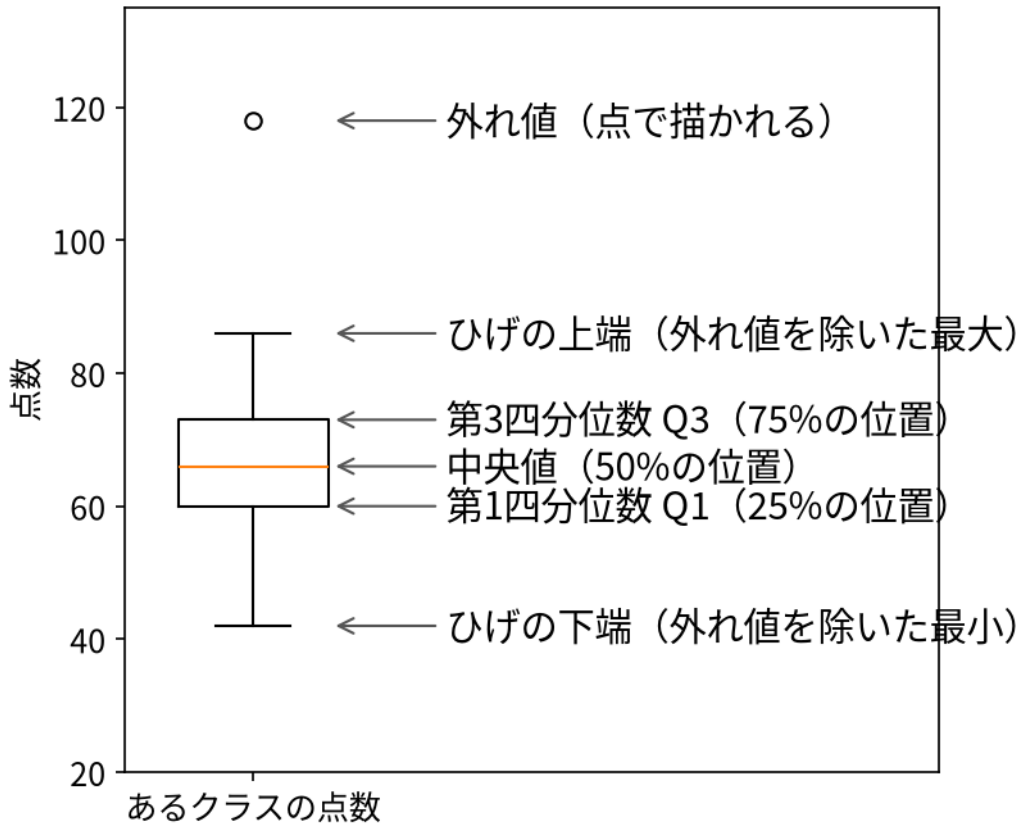


図 6.1: 箱ひげ図の解剖図

6.6.2　箱ひげ図の描き方

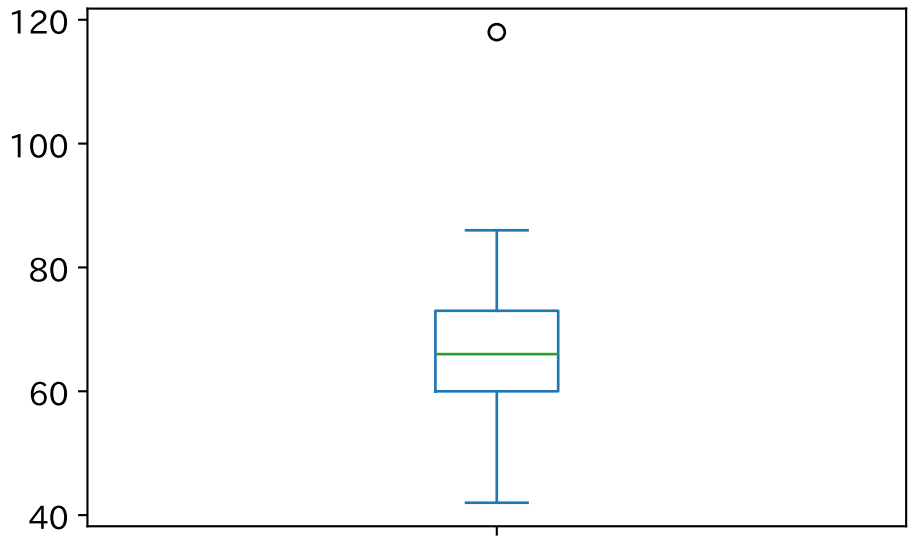
1. データを小さい順に並べ、四分位数（Q1・中央値・Q3）を求める
2. Q1 から Q3 までを箱で囲み、中央値に線を引く（太線）
3. 「ひげ」は、箱から 1.5×IQR 以内にある最も端のデータまで伸ばす
4. それより外の点は、1 つずつ点で描く（外れ値）

6.6.3　外れ値（ハズレ値）：飛び抜けた点の扱い

- 箱ひげ図で、ひげの外に点で描かれる値が外れ値です
- 数式でも定義されます：Q1 − 1.5×IQR より小さい、または Q3 ＋ 1.5×IQR より大きい
- 外れ値も大事な情報なので，削除して無かったことにしてはいけません．

```
one_class = pd.Series([42, 48, 52, 55, 57, 58, 60, 61, 62, 63, 64,
 ↪  65, 66,
                       67, 68, 69, 70, 71, 73, 75, 77, 79, 82, 86,
                        ↪  118])
q1 = one_class.quantile(0.25)
med = one_class.median()
q3 = one_class.quantile(0.75)
iqr = q3 - q1
print(f"Q1={q1}, 中央値={med}, Q3={q3}, IQR={iqr}")
lo, hi = q1 - 1.5 * iqr, q3 + 1.5 * iqr
print("外れ値の候補:", one_class[(one_class < lo) | (one_class >
 ↪  hi)].tolist())
plt.figure()
one_class.plot.box()
plt.show()
```

```
Q1=60.0, 中央値=66.0, Q3=73.0, IQR=13.0
外れ値の候補: [118]
```

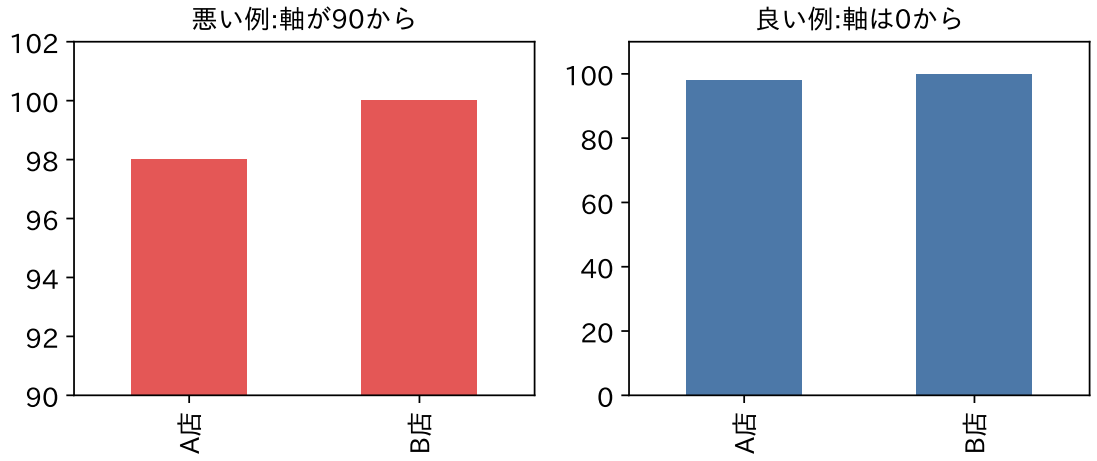


6.7　伝わるデザインの約束

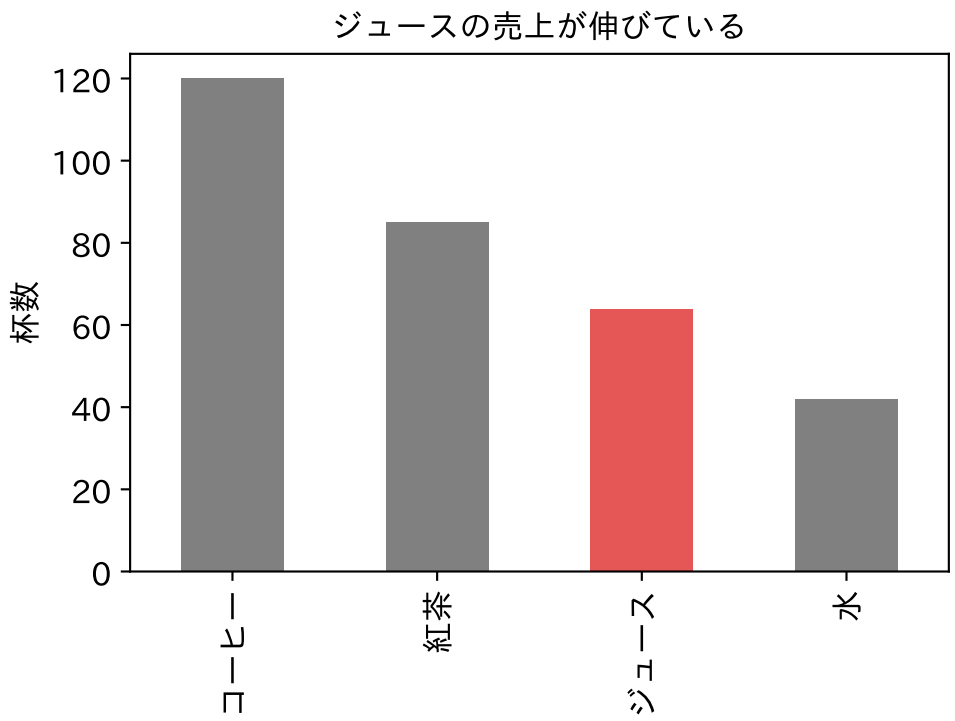
- 軸は原則 **0** から（途中から始めると、差が実際より大きく見えます）

- タイトルは「何の図か」＋できれば**主張を一言**
- 単位と件数を添える
- 色は少なく。強調したい所だけ色を付ける
- 飾り（影、立体、グラデーション）は付けない

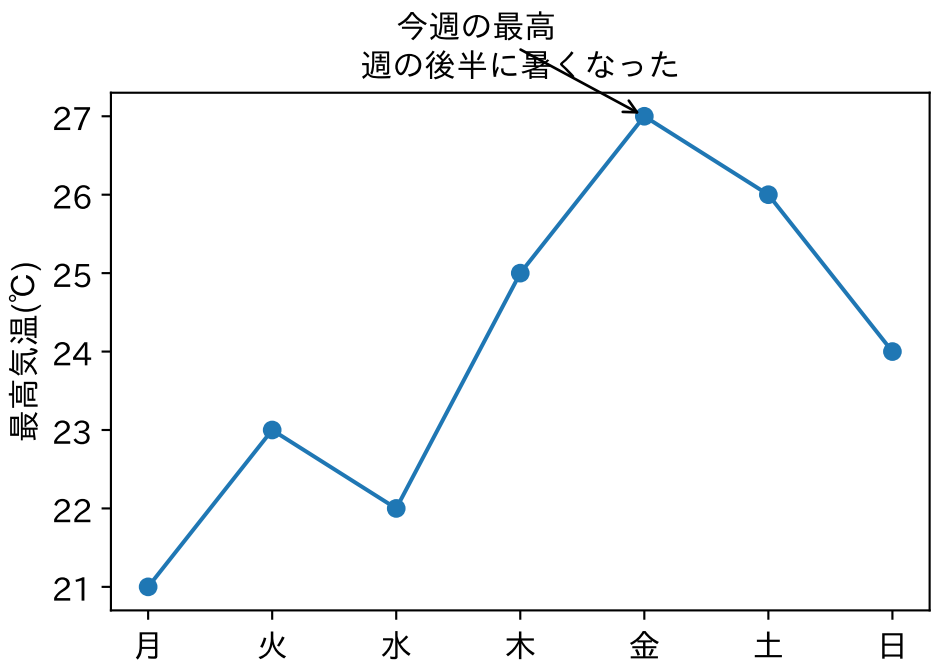
```
vals = pd.Series([98, 100], index=["A店", "B店"])
fig, axes = plt.subplots(1, 2, figsize=(7.2, 3.2))
vals.plot.bar(ax=axes[0], color="#E45756", ylim=(90, 102), title="
 ↪  悪い例:軸が90から")
vals.plot.bar(ax=axes[1], color="#4C78A8", ylim=(0, 110), title="
 ↪  良い例:軸は0から")
plt.tight_layout()
plt.show()
```



```
sales.plot.bar(color=["gray", "gray", "#E45756", "gray"],
               title="ジュースの売上が伸びている", ylabel="杯数")
plt.show()
```

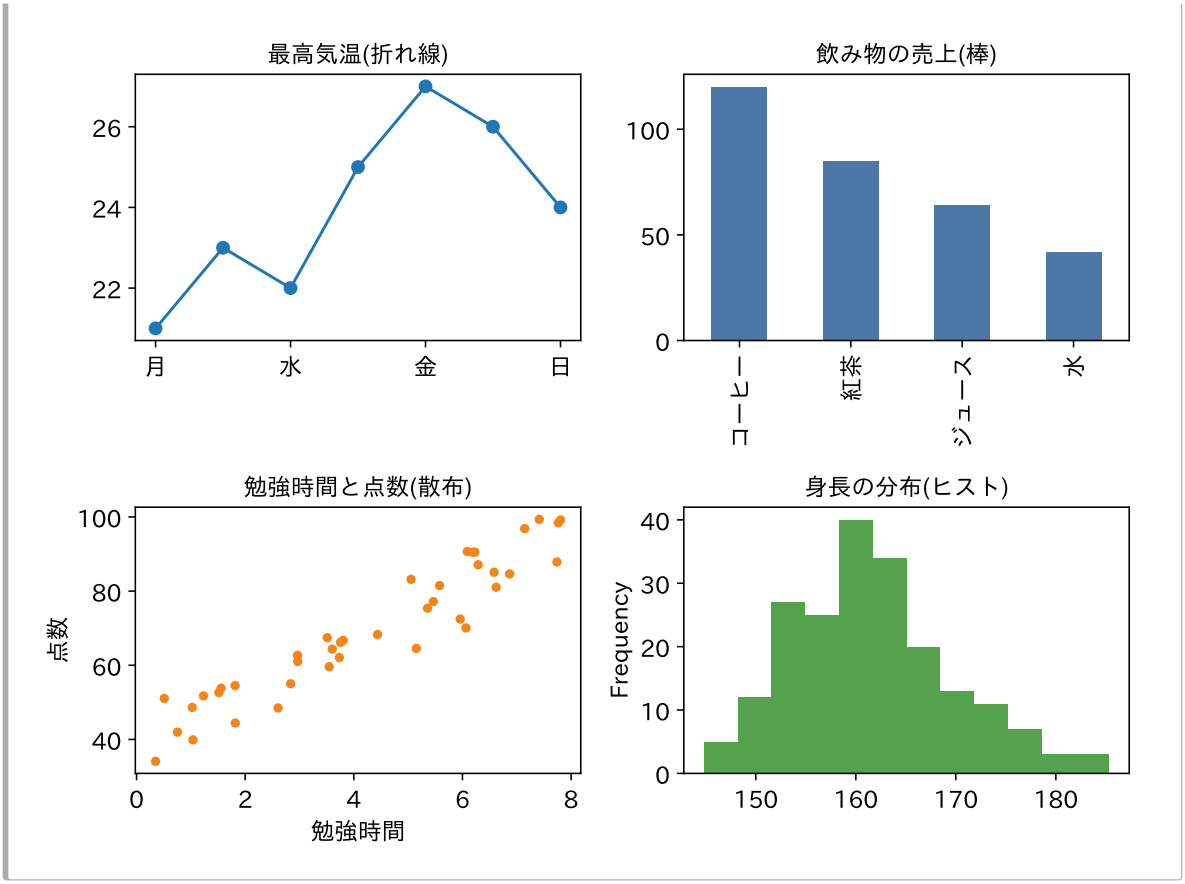


```
ax = temp.plot.line(marker="o", title="週の後半に暑くなった",
 ↪  ylabel="最高気温(℃)")
ax.annotate("今週の最高", xy=(4, 27), xytext=(2, 28),  # 位置は左から
 ↪  数えた番号(月=0…金=4)
            arrowprops=dict(arrowstyle="->"))
plt.show()
```



6.8　並べて比べる

```
fig, axes = plt.subplots(2, 2, figsize=(8, 6))
temp.plot.line(ax=axes[0, 0], marker="o", title="最高気温（折れ線）")
sales.plot.bar(ax=axes[0, 1], color="#4C78A8", title="飲み物の売上
↪ （棒）")
study.plot.scatter(ax=axes[1, 0], x="勉強時間", y="点数", s=14,
                   color="#F58518", title="勉強時間と点数（散布）")
heights.plot.hist(ax=axes[1, 1], bins=12, color="#54A24B",
                  title="身長の分布（ヒスト）")
plt.tight_layout()
plt.show()
```



6.9 演習

1. お題(出席率の変化／満足度の偏り／身長と体重の関係)に合う形を描きましょう
2. 軸が 90 からの棒グラフを、0 からに直しましょう
3. タイトルを「結論を書く形」に書き換えましょう

6.10 前半のまとめ(グラフのいろいろ)

- グラフは「知りたいこと」で選ぶ。各グラフは「何のどんな様子」を見せるためのものか言えること
- 箱ひげ図は四分位数の図。外れ値は、理由を調べてから扱う
- pandas の .plot で描き、matplotlib で仕上げる
- デザインの約束：軸は 0 から、タイトル＝主張、単位と件数、色は少なく

6.11　データ分析：気づきをグラフだけで納得させる

後半は、実際のデータ（反応時間の実験）を眺めて、気づきをグラフだけで納得させる実習です。講師デモを見てから、一緒に実習し、最後に演習をします。

6.12　分析の進め方：眺める → 気づく → 確かめる →納得させる

1. **眺める**：一覧表と要約の表を、しばらく眺める
2. **気づきをメモする**：「音楽があると遅そう」「年代で違いそう」のように、仮の言葉でよい
3. **集計で確かめる**：一元表・クロス集計で、気づきが数字の上でも言えるか見る
4. **グラフに仕上げる**：タイトル＝主張の 1 枚で、説明が無くても伝わる形にする

6.13　比べるときは「分布の格差」で見る

- このコマでの比較は、平均の差だけでは決めません。**分布の格差（中心・広がり・重なり）**もとに行います
  - 中心：中央値（箱の線）がどれくらいずれているか
  - 広がり：箱やひげの幅が、グループでどれくらい違うか
  - 重なり：2 つの分布がどれくらい重なっているか（重なりが大きいと「違う」と言いにくい）
- 箱ひげ図は、この 3 つが一度に読めるので比較に向きます

6.14　実習データ：反応時間の実験（80 名）

- 列：ID、年代、条件（静か／音楽あり）、反応時間（ミリ秒）、正答数（20 問中）（psych_rt.csv）
- 画面に合図が出たらボタンを押す、という課題を 80 名に実施した記録です
- まず一覧を眺め、条件ごと・年代ごとの要約の表を作ってみましょう

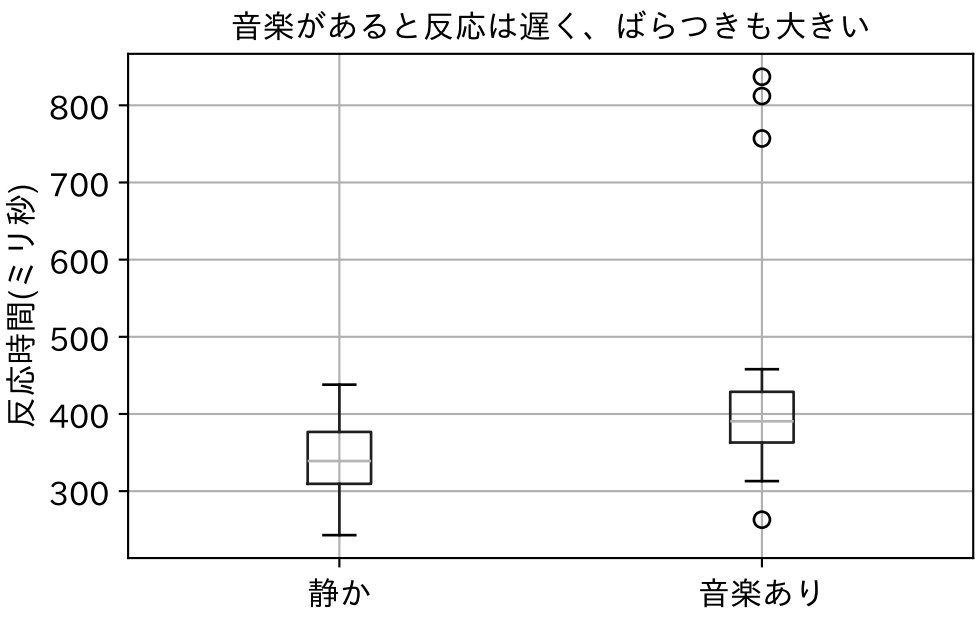
```
rt = pd.read_csv("data/psych_rt.csv")
display(rt.head(10))
display(rt.groupby("条件")["反応時間"].agg(["mean", "median",
↪  "std", "count"]))
```

	ID	年代	条件	反応時間	正答数
0	1	60 代	音楽あり	432	19
1	2	20 代	音楽あり	315	15
2	3	20 代	音楽あり	353	12
3	4	60 代	音楽あり	451	19
4	5	40 代	音楽あり	428	11
5	6	20 代	静か	350	19
6	7	40 代	音楽あり	387	17
7	8	40 代	音楽あり	370	14
8	9	40 代	音楽あり	363	13
9	10	40 代	音楽あり	363	13

条件	mean	median	std	count
静か	340.925	339.0	49.819874	40
音楽あり	416.325	390.5	119.416184	40

平均と中央値がずれる条件がある → 外れ値の気配があります。

```
rt.boxplot(column="反応時間", by="条件")
plt.title("音楽があると反応は遅く、ばらつきも大きい")
plt.suptitle("")
plt.xlabel("")
plt.ylabel("反応時間（ミリ秒）")
plt.show()
```



6.15　「グラフだけで納得させる」ための工夫

- タイトルを主張そのものにする（「音楽ありのほうが遅い」）
- 比べる相手を同じ図の中に置く
- 強調色と注釈で、見てほしい場所を指す
- 説明文を添えなくても主張が伝わるか、隣の人に見せて確かめる

6.16　演習

1. 反応時間のデータから、自分の「気づき」を **1** つ選ぶ
2. それを、タイトル＝主張のグラフ 1 枚で表す。説明の文章は書かない。比べる場合は、平均だけでなく分布を並べた図にすること
3. 隣の人と見せ合い、主張が一目で伝わるか確かめ合う

6.17　まとめ

- 分析は「眺める→気づく→確かめる→グラフで納得させる」の順
- 比較は、平均ではなく分布の格差（中心・広がり・重なり）で見る
- グラフだけで納得させるには、タイトル＝主張と比較相手が要る

第 7 章

第 7 コマ　データスクレイピングと図表作成

このコマの進め方 :講義とデモ(30 分) → 一緒に実習(20 分) →班の実習(30 分) → 発表とまとめ(10 分)。

7.1　データスクレイピングとは

- スクレイピングとは、ウェブページで公開されている表などのデータを、プログラムで集める方法です。第 5 コマで見た「データ集めの主な方法」の一つです
- pandas には、ウェブページの表を取り込む `read_html` という道具があります
- ここでは実際に、Wikipedia の「長崎市」のページから、国勢調査による人口推移の表を取り込んで、グラフにしてみます

まず、ページを取りに行きます。`requests` という道具でページをもらってから、`read_html` に渡します

本資料(画像を含む)の無断転載・無断使用は禁止します.

```
import requests
from io import StringIO

url = "https://ja.wikipedia.org/wiki/長崎市"
html = requests.get(url, headers={"User-Agent": "Mozilla/5.0
 ↪  (lecture practice)"}).text
tables = pd.read_html(StringIO(html))
print(len(tables), "個の表が見つかりました")
```

```
20 個の表が見つかりました
```

- `headers={"User-Agent": ...}` の 1 行は、「ブラウザからのアクセスです」という名乗りです。プログラムからのアクセスは、名乗りがないと断られてしまうサイトがあります（このページも、名乗りなしでは取り込めませんでした）
- 取り込んだ表は `tables` に全部入っています。このページには表がたくさんあり、その数はページの編集で変わることもあります

表の番号は変わりうるので、「何番目の表」と決め打ちせず、**列の名前で目的の表を探す**のが安全です

```
# 「長崎市人口」という列がある表を探す
pop = None
for t in tables:
    if any("長崎市人口" in str(c) for c in t.columns):
        pop = t
        break
display(pop[["実施年", "長崎市人口(人)"]])
```

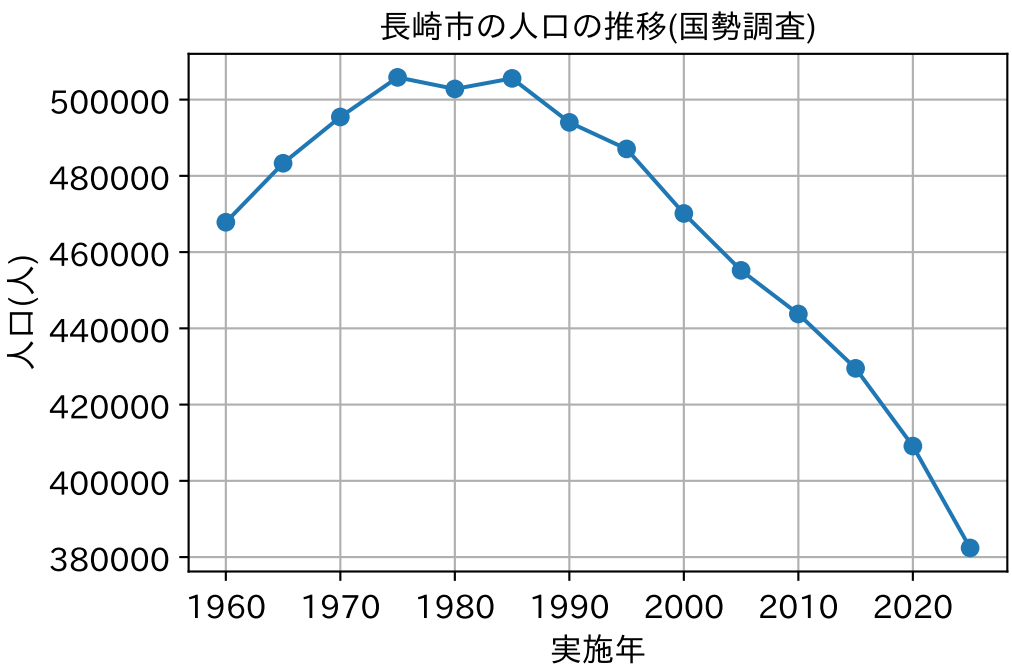
	実施年	長崎市人口 (人)
0	1960 年	467835
1	1965 年	483303
2	1970 年	495445
3	1975 年	505835

4	1980 年	502799
5	1985 年	505566
6	1990 年	494032
7	1995 年	487063
8	2000 年	470135
9	2005 年	455206
10	2010 年	443766
11	2015 年	429508
12	2020 年	409118
13	2025 年	382393

必要な 2 列を取り出し、「1960 年」のような文字になっている実施年を数値に直して、折れ線グラフを描きます

```
pop2 = pop[["実施年", "長崎市人口(人)"]].copy()
pop2["実施年"] = pop2["実施年"].astype(str).str.replace("年",
 ↪  "").astype(int)

plt.plot(pop2["実施年"], pop2["長崎市人口(人)"], marker="o")
plt.title("長崎市の人口の推移(国勢調査)")
plt.xlabel("実施年")
plt.ylabel("人口(人)")
plt.grid(True)
plt.show()
```



- このグラフは、Wikipedia「長崎市」に載っている、国勢調査による人口推移の表をもとに作成しました。スクレイピングで作った図表には、このように**出典を書く**習慣をつけましょう
- 公開されているデータでも、サイトの利用条件や著作権を確認し、出典を記録しておくことが約束です。短時間に何度もアクセスしすぎないことも大切です
- 当日ネットにつながらないときは、配布データの `data/nagasaki_population.csv` に同じ表(実施年と人口の 2 列)が入っています。`pd.read_csv("data/nagasaki_population.` で読み込めば、グラフを描く練習は同じようにできます
- このコマでは、スクレイピングで集めたデータで図表を作り、読み取れたことを発表するところまでを体験します

7.2　発展：年齢層の割合の推移

- 同じ手順で、全国のデータも取れます。Wikipedia「日本の人口統計」には、年ごとの年齢層割合(0〜14 歳／ 15〜64 歳／ 65 歳以上)の表があります
- この表は列の名前が 2 段組になっていて、そのままでは扱いにくいので、列名を整える工夫が要ります。本編より少し難しいので、発展として紹介します

```
url2 = "https://ja.wikipedia.org/wiki/日本の人口統計"
html2 = requests.get(url2, headers={"User-Agent": "Mozilla/5.0
 ↪  (lecture practice)"}).text
tables2 = pd.read_html(StringIO(html2))

age = None
for t in tables2:
    if any("年齢層割合" in str(c) for c in t.columns):
        age = t
        break
display(age.head())
```

	Year	人口（千人）	年齢層割合 (%)		
	Year	人口（千人）	0–14	15–64	65+
0	1935	69254	36.9%	58.5%	4.7%
1	1940	73114	36.1%	59.2%	5.7%
2	1945	71998	36.8%	58.1%	5.1%
3	1950	83199	35.4%	59.6%	4.9%
4	1955	89275	33.4%	61.2%	5.3%

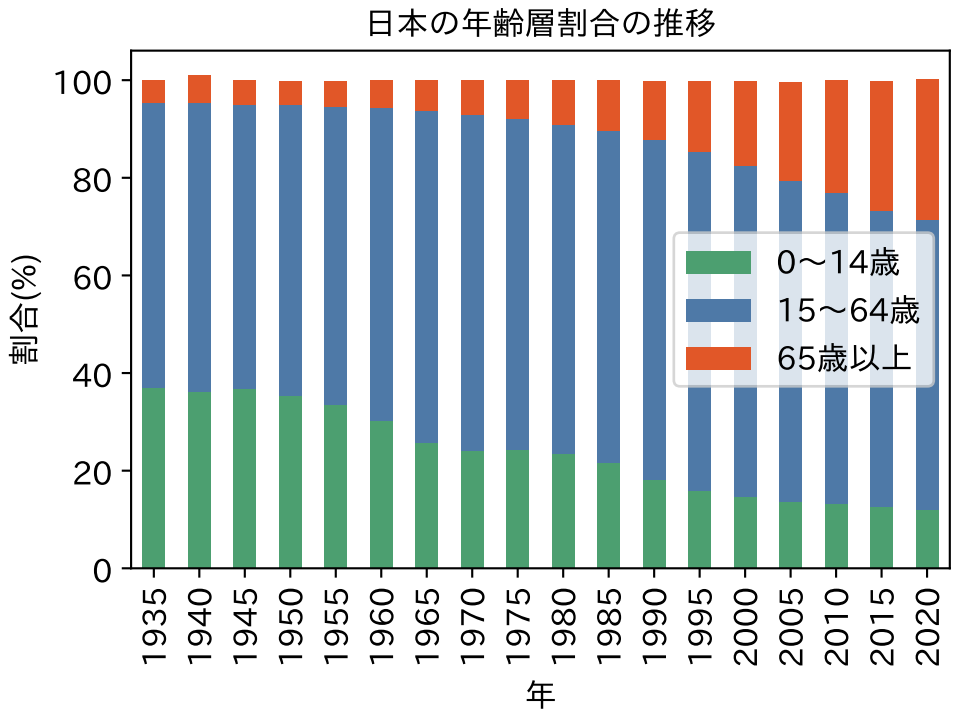
```
# 2段になっている列名を、下の段の名前だけにそろえる
age = age.copy()
age.columns = [c[-1] if isinstance(c, tuple) else c for c in
 ↪  age.columns]

age2 = age[["Year", "0-14", "15-64", "65+"]].copy()
age2.columns = ["年", "0〜14歳", "15〜64歳", "65歳以上"]
for col in ["0〜14歳", "15〜64歳", "65歳以上"]:
    age2[col] = age2[col].astype(str).str.replace("%", "",
 ↪  regex=False).astype(float)
display(age2.head())
```

	年	0〜14 歳	15〜64 歳	65 歳以上
0	1935	36.9	58.5	4.7
1	1940	36.1	59.2	5.7
2	1945	36.8	58.1	5.1
3	1950	35.4	59.6	4.9
4	1955	33.4	61.2	5.3

割合の内訳の移り変わりは、積み上げ棒グラフにすると見やすくなります

```
age2.plot(x="年", kind="bar", stacked=True,
          color=["#4C9F70", "#4E79A7", "#E15728"])
plt.title("日本の年齢層割合の推移")
plt.xlabel("年")
plt.ylabel("割合(%)")
plt.legend()
plt.show()
```



- 緑が 0〜14 歳、青が 15〜64 歳、赤が 65 歳以上です。年代が進むほど赤の部分が大き

くなり、高齢化が進む様子が一目で分かります
- これは長崎市ではなく**全国**のデータです。出典：Wikipedia「日本の人口統計」の年齢層割合の表をもとに作成しました

7.3 集めたデータの整え方と図表の作り方

- 取り込んだ表は、そのままではグラフにできないことが多いです。デモでやった整え方をまとめると、次の 4 つです
  1. 目的の表を**探す**（何番目かではなく、列の名前で探す）
  2. 必要な列だけを**取り出す**
  3. **型を直す**（「1960 年」→ 1960、「36.9%」→ 36.9 のように、文字を数値へ）
  4. 列に**分かりやすい名前**を付け替える
- グラフ選びは第 6 コマの要点と同じです。時間の変化なら折れ線、量の大小なら棒、全体の中の内訳の移り変わりなら積み上げ棒。まず「何のどんな様子を見せたいか」を一言で言ってから選びます
- 仕上げに、タイトルと軸の名前を付け、出典を記録します

7.4 実習

まず「一緒に実習」では、上のデモのコードを自分のノートブックでも実行して、長崎市の人口グラフを再現してみましょう（ネットにつながらないときは `data/nagasaki_population.csv` を読み込んでください）

班の実習では、テーマを決めて、自分たちで表を取りに行きます

1. 班ごとにテーマを 1 つ決める（例：自分の出身地の人口の移り変わり、県庁所在地の気候、人口と面積の関係など）
2. テーマに合う表のあるウェブページを探す（Wikipedia の自治体のページには、人口や気候の表が載っていることが多いです）
3. デモと同じ手順で表を取得し、整えて、図表を 1 つ作る
4. 図表から読み取れた「気づき」を 1 つ、発表する

発表では、どのページのどの表を使ったか（出典）もあわせて紹介してください

7.5　まとめ

- スクレイピングでは、ウェブで公開された表をプログラムで取り込める。サイトによっては名乗り（User-Agent）が必要
- 取り込んだ表は「探す→取り出す→型を直す→名前を付け替える」と整えてから、目的に合ったグラフにする
- 図表には出典を記録する。短時間に何度もアクセスしないなど、集める側のマナーも大切
- 次のコマ（まとめ）では、各自、データから気づきをグラフで納得させる課題に取り組みます

第 8 章

第 8 コマ　まとめ

このコマの進め方 :講座全体の振り返り（10 分） → まとめの課題（70 分） →振り返り（10 分）。講座のまとめのコマです。第 1〜7 コマの道具を、自分の力で使います。

8.1　講座全体の振り返り

- このコマは、講座全体の**まとめ**です
- 2 日間で学んだ流れは、「データから表へ、表からグラフへ」でした。表の作り方（第 4 コマ）、尺度と型（第 5 コマ）、グラフの選び方と分析（第 6 コマ）、スクレイピングでの集め方（第 7 コマ）を一通り使います
- 「平均だけでは分からない姿を、グラフで知る」——これがこの講座の芯でした（第 1 コマ）
- 最後に、まとめの課題として、自分の力で「気づきをグラフで納得させる」テストに取り組みます

8.2　まとめの課題（テスト）

配布されたデータ（test_psych.csv :ある注意課題の実験に参加した 120 名の記録）を眺めて、気づきを **1 つ**挙げ、それを**グラフだけで納得させる**形で示しなさい。

- 列：ID、年代、性別、条件（単独／ペア）、時間帯（午前／午後）、反応時間（ミリ秒）、正答数（20 問中）
- 提出物：
  - ①気づき（1 文）
  - ②その気づきを納得させるグラフ（タイトル＝主張）

本資料（画像を含む）の無断転載・無断使用は禁止します.

  - ③使ったプログラム
- 比べる場合は、平均だけでなく**分布の格差（中心・広がり・重なり）**が見える形にすること
- 制限時間 70 分

8.3　課題スターター

```
df = pd.read_csv("data/test_psych.csv")
display(df.head())
display(df.describe(include="all"))
# ここから先は自分で書く
```

	ID	年代	性別	条件	時間帯	反応時間	正答数
0	1	20 代	女性	ペア	午前	295.0	17
1	2	20 代	女性	ペア	午後	311.0	13
2	3	20 代	女性	単独	午前	339.0	16
3	4	40 代	男性	単独	午後	343.0	18
4	5	40 代	女性	ペア	午後	369.0	20

	ID	年代	性別	条件	時間帯	反応時間	正答数
count	120.000000	120	120	120	120	118.000000	120.000000
unique	NaN	3	2	2	2	NaN	NaN
top	NaN	60 代	女性	ペア	午前	NaN	NaN
freq	NaN	44	63	60	60	NaN	NaN
mean	60.500000	NaN	NaN	NaN	NaN	391.940678	15.758333
std	34.785054	NaN	NaN	NaN	NaN	101.312776	2.376339
min	1.000000	NaN	NaN	NaN	NaN	210.000000	9.000000
25%	30.750000	NaN	NaN	NaN	NaN	339.000000	14.000000
50%	60.500000	NaN	NaN	NaN	NaN	377.000000	16.000000
75%	90.250000	NaN	NaN	NaN	NaN	422.500000	17.000000
max	120.000000	NaN	NaN	NaN	NaN	1050.000000	20.000000

8.4 進め方の目安(第 6 コマの手順)

1. **眺める**(15 分):一覧と describe、groupby で一通り眺める
2. **気づきを 1 つ選ぶ**(5 分):一文で言える形にする
3. **確かめる**(15 分):集計で数字の裏づけを取る。比べる場合は分布の格差を見る
4. **グラフに仕上げる**(25 分):タイトル＝主張、軸は 0 から、単位と件数
5. **見直す**(10 分):下のチェックリストで自己確認して提出

8.5 提出前チェックリスト

- □ 気づきは一文で書いたか
- □ グラフのタイトルが、主張になっているか
- □ 比べる相手が同じ図に入っているか
- □ 軸が 0 から始まっているか(量のグラフの場合)
- □ 単位と件数(n ＝)を書いたか
- □ プログラムが、もう一度実行しても同じ図になるか

8.6 振り返り

- お互いのグラフを見て、「一目で主張が伝わるか」を確かめ合います
- 「平均だけでは分からない姿を、グラフで知る」——これがこの講座の芯でした(第 1 コマ)

用語のミニ辞典

- **JupyterLab Desktop**：ノートブックを使うためのソフト。Python 本体が一緒に入っている
- **セル**：ノートブックの中で、コードを書く欄。セルごとに Shift+Enter で実行する
- **カーネル**：ノートブックの裏で Python を実行している中身。再起動すると、作った変数が全部消える
- **変数（統計）**：測ったり記録したりする項目。尺度（名義・順序・間隔・比）がある
- **DataFrame**：pandas の表全体。Excel のシートのようなもの
- **集計**：データを種類ごとに分けて合計・平均などを出すこと
- **分布**：値のばらつき具合。ヒストグラムや箱ひげ図で見る
- **母集団と標本**：本当に知りたい全体が母集団、実際に集めたデータが標本（サンプル）
- **混同行列**：推測の正誤の数を数える表。行と列が同じカテゴリで、斜めが正解
- **度量衡**：長さ（度）・かさ（量）・重さ（衡）を測る、昔からの共通のものさし・単位の体系
- **分割表**：クロス集計表の別名。2 つの変数の組み合わせで人数などを数える

本資料（画像を含む）の無断転載・無断使用は禁止します.

困ったときの対処表

症状	対処
真っ赤なエラーが出た	セルの下のメッセージの一番下の行を読む。かっこ・スペル・全角文字を確認
グラフが表示されない	`plt.show()` の書き忘れ。直して、そのセルをもう一度実行する（Shift+Enter）
日本語が四角になる	matplotlib-fontja を使う（第 5 コマ）。**ノートブックで使う Python（カーネル）に入っていないと効かない**ので、第 3 コマの `%pip install` を実行したか確認する。教材のコードは、matplotlib-fontja が使えない場合に入っている日本語フォント（IPAex ゴシック／ Noto Sans CJK ／游ゴシック等）を自動で探す
変数が無いと言われる	セルは実行した順に効く。必要なセルが実行済みか確認する。カーネルを再起動した後は、上のセルから順に実行し直す
セルを実行しても進まない・反応がない	カーネル（Python を実行している中身）が止まっていることがある。メニューの「Kernel → Restart Kernel」で再起動し、上のセルから実行し直す
CSV が文字化け・見つからない	`encoding` の指定／ data フォルダが mylab の中にあるか、ノートブックが mylab の直下にあるか確認
保存した図が見当たらない	`savefig` は `show` の前。同じフォルダを探す

本資料（画像を含む）の無断転載・無断使用は禁止します.